

Kimi K3 技术报告领读 · 学习文档

来源：论文学习播客《Kimi K3 技术报告领读》逐字稿（时长 2h04min）

主持人：小俊 / 另一位主持 (Speaker 3)

主讲嘉宾：孙宇韬，清华大学计算机系博士候选人，上海创智学院璞瑞学者

嘉宾研究方向：LLM 架构与预训练，核心关切是"推理高效性"

本文档定位：不是论文摘要，而是把 K3 的每一个设计点还原到它背后的研究脉络——串联了

RetNet、YOCO、Mamba、DeltaNet、Gated DeltaNet、Kimi Linear/KDA、

HyperConnection、DenseNet、Latent MoE、MiniLM/OPD、EAGLE-3 等十余篇工作。

目录

- 0. 本次分享的方法论
- 1. 嘉宾的研究脉络（理解 K3 的前置知识）
- 2. K3 总览：三个维度的有效 Scaling
- 3. 模型架构（本次分享的重头戏）
 - 3.1 线性注意力发展史 → Kimi Delta Attention (KDA)
 - 3.2 Gated MLA
 - 3.3 Attention Residual：从 ResNet 到 DenseNet 到 HyperConnection
 - 3.4 Latent MoE
 - 3.5 C2GLU：给 MLP 中间激活加数学上界
 - 3.6 优化器：Muon 一脉的稳定性工程
 - 3.7 Quantile Balancing：MoE 负载均衡的"更 principled"解法
 - 3.8 Vision Encoder：为什么 from scratch
 - 3.9 为什么 K3 不用 Sparse Attention
- 4. 预训练
- 5. 后训练
- 6. Infra
- 7. 观点与访谈精华
- 8. 术语勘误表与延伸阅读

0. 本次分享的方法论

嘉宾没有按论文章节顺序照本宣科，而是采用了自己惯用的讲法：

"所有的论文都是建立在之前无数个 related work 的基础之上。如果只看这篇论文，你会知道他是这样做的，但你可能会不太清楚他**为什么**这样做，或者说历史上大家是怎么做的，为什么最后达到了这个解。"

因此本次分享的结构是：选取论文中几个关键贡献点 → 把每个点背后的历史脉络打开 → 回到 K3 看它最终的选择。

为什么架构章节占篇幅最大：

- 1. K3 本身在模型架构上做了比较多的创新（这是它与 K2 最大的不同）；
- 2. 模型架构这条线的研究脉络相当清晰，而且每一部分都有比较明确的 credit 归属，可以拆成一个个点讲清楚。

哪些部分讲得少：

- **Pretrain data**：各家都是机密状态，paper 里也没写。嘉宾明确表示"不会讲 paper 公开内容以外的东西"。
- **Post-train 具体 RL task**：偏工程、偏具体场景，论文写得也不多。
- **Infra 会讲**，因为现在架构设计高度依赖 infra 与模型的 co-design，两者密不可分。

一个重要的价值观：主持人指出，最终看到的是 K3 这一个呈现，但里面有很多人的 credit，不只是 Kimi 一家公司。嘉宾对此的评价是：

"作为一个公司，比较科学的方式就是如果你最后想把这个模型做好，你肯定是集百家之长。如果你只想用自己的工作，那你就会错过一些别人做得比较好的工作。整个行业都是由大家集体推动的。K0、K1、K2、K3 一个比较好的点，就是他会比较客观、比较真实地把之前的一些工作用可靠的方式引出来，不会想办法去削弱之前一些人的贡献。"

1. 嘉宾的研究脉络（理解 K3 的前置知识）

嘉宾从 2023 年开始做大模型架构与预训练，核心问题是降低大模型推理环节的低效性。他先给出了一个重要的范式判断：

1.0 范式变化：从"性能导向"到"效率导向"

时代	架构研究的目标
ImageNet / Vision 时代	设计更精妙的架构，直接提升模型本身的表现
大模型时代	架构改进对性能的提升极其微小；但不同架构在推理效率上差异巨大

关键结论（贯穿全场的第一性原理）：

对于模型性能而言，模型的参数量永远是最主要的因素。架构本身的改进相对于参数量提升带来的性能收益是相当微小的。但不同架构在模型推理方面性能差异是巨大的，而且这个主要决定了模型最后部署的价格。

因此这两年整个业界的架构研究方向，是从"提性能"转向"提效率"。

1.1 第一篇工作：RetNet（2023）

△ 逐字稿中被 ASR 识别成"ResNet"，实为 RetNet (Retentive Network)。

当时线性注意力的处境：线性注意力最初提出，是为了用一个 Linear Kernel、以 kernelized 的方式去拟合二次复杂度的全注意力。在那个时代，线性注意力对位置信息不够 **sensitive**。虽然线性注意力和 RNN 在形式上相似，但从建模特性上讲，RNN 更强调 locality 的表示。

RetNet 解决了两个问题：

(1) 引入衰减 (decay) 机制

最原始的线性注意力就是"KV 乘出来得到一个外积，然后加起来"——比 RetNet 还要简单，而且是 **position invariant** (位置无关) 的。

让线性注意力变成位置有关，有两条路：

- 加 RoPE
- 引入衰减项

RetNet 选择了衰减。动机是：**语言的特性就是更近的位置有更高的权重**。在一个有限的状态空间（有限 KV cache）里，衰减能让模型尽量去建模"对最后结果贡献足够大"的信息。

这一项后来被几乎所有线性注意力工作保留了下来。

(2) 提出 chunkwise recurrent 计算形式 ← 影响更深远

这是一个 **infra** 层面的创新。问题在于：

计算形式	问题
完全递归（token 维度逐步递推）	效率不是最优，训练时甚至比 parallel representation （类全注意力）还慢
Parallel representation（仿照全注意力算）	享受不到线性复杂度的优势；训练时计算强度和全注意力一模一样甚至更高（kernel 可能还写不过 FlashAttention）

于是在两者之间找了一个 trade-off: **chunkwise recurrent** (块递归) 。

- **好处 1：**拿到整体计算复杂度上的收益；
- **好处 2：**在 kernelize 层面尽量多地调用 TensorCore，把 local 的计算密度打满。

这是本次分享中最重要的**"基础设施"概念之一**。之后所有线性注意力的改进工作——Mamba2、Gated DeltaNet，直到今天的 Kimi Linear / KDA——都是基于 **chunkwise** 的计算形式。后续创新都是在这个大计算范式之上的细节变化。

RetNet 作为"纯线性注意力"的结论：从今天的视角看，**纯线性注意力无疑是一次比较失败的尝试**。有限的 state 无论如何都难以在长上下文中达到与全注意力完全相同的性能——这在今天已是共识。

1.2 转向 Hybrid Attention (2023 年即意识到)

意识到纯线性的天花板后，嘉宾转向 **hybrid attention**：把线性注意力和全注意力组合起来（K3 也是这么做的）。

一个关键的实验性结论（今天已是公认）：

混合注意力从架构上讲是一种 trade-off，但从最后模型本身的表现来说，并不是一个 **trade-off**。在保持一定全注意力比例的基础上，整个模型可以获得**无损甚至更好的**长上下文表现。也就是说，我们并没有牺牲全注意力的模型能力来换取工程收益。

正是这个结论，导致混合注意力现在被大规模利用起来。

但它有个让博士生“不够满意”的问题：

混合注意力的推理加速比，永远和混合比成正比例。目前常用的混合比是 **线性:全注意力 = 3:1**（全注意力占 1/4）。如果 1/4 是无损的极限比例，那么无论 prefill 还是 decode，**最多也就 4 倍加速比**——这只是一个常数级别的改进。

1.3 第二篇工作：YOCO (You Only Cache Once)

第一性原理拆解：为什么纯线性注意力无法获得和全注意力一样的性能？因为我们至少期望模型有“获取前文信息”的基本能力，所以长上下文的 **KV cache** 这一点没法省掉。

既然 token 长度维度省不掉，那就**换一个维度省**——从层间省。

YOCO 的核心设计：

1. **所有层共用一份 KV cache**。从 KV cache 存储角度这已经是极限了——如果从 token-wise 角度必须保留，那“一份”就是最少的，不可能再少。
2. **把模型的计算和存储解耦**：虽然只有一份 KV cache，但可以保持**多层的 full attention 计算结构**，从而获得与全注意力/混合注意力基本等价的计算结果。
3. **Prefill 阶段可以直接跳过全注意力的计算**。因为 prefill 的唯一目的是拿到 KV cache（用于后续 decode），并不需要解码出每个位置的 next token prediction。所以 prefill 时经过前面的 linear attention 拿到 KV cache 就够了，可以直接不过 cross attention；而 decode 阶段保持和传统模型一样的计算能力。

标题致敬 YOLO (You Only Look Once)，后来也有一系列致敬的工作。

1.4 推理开销的三分法（重要框架）

模型推理开销只有三个部分，不可能有第四个：

1. **Prefill 的时间开销**
2. **Decode 的时间开销**
3. **KV cache 的存储开销**

- YOCO 解决了 **prefill** 和 **KV cache**；
- 没有解决 **decode**；
- decode 后续大家尝试用 **sparse attention** 解决——但 **K3** 目前没有用 **sparse attention**（原因见 3.9）。

1.5 第三篇工作：YOCO-Universal (Loop Language Model 方向)

Loop Language Model 是什么：在保持固定模型参数大小的情况下，通过多次迭代来提升模型 FLOPs，从而提升总能力。目标是用较小参数量取得比同参数模型更好的效果。

传统 Loop LM 的两个致命问题：

1. **只省参数不省计算。**在相同计算量下，它其实不如"把参数展开"或"用一个更大但不 loop 的模型"效果好。Decode 成本难以忍受。
2. **Cache 随推理深度增加。**最后总的 cache 相当大。

YOCO-Universal 的解法（在 YOCO 架构基础上做位置改变）：把 **loop 单元只放在线性注意力这一段**。

这样做的原因是 YOCO 的混合方式**不是交错式 (interleave) 的，而是"两段式"**：模型前期是全线性注意力，后期是全注意力。所以：

- **省存储：**Linear attention 本身的 cache 开销微乎其微，loop 它对整体 KV 增长影响很小；
- **省计算：**在长上下文场景下，linear attention 带来的额外计算相当小；
- **保能力：**虽然 linear attention 的 decode 计算很小，但从 perplexity 和 scaling behavior 看，它能获得足够接近全注意力的模型能力。

效果：用约 2× 的计算强度，获得约 2× 的性能提升，而**模型存储和 KV cache 与原模型基本持平**。

1.6 为什么一直做架构创新

个人层面：

"架构创新比较有意思、比较好玩。别的层面的工作工程性因素更强。当然架构创新也有工程性，但它的工程性主要来源于你需要做 infra 上的 co-design——而 infra 上的 co-design 本身也是创新（比如 chunkwise recurrent）。而且它比较能起到**四两拨千斤**的效果。对一个博士生而言，这是一个相当好的研究领域。"

行业层面：

"全注意力当时是一个非常大的瓶颈。如果大模型要大规模应用，attention 这个东西是一定需要解决、而且我们相信一定可以解决的。所以在那个时间点，架构创新对于模型部署而言是最重要的事情。"

△ **一个重要的时代判断：**

"在 **2023 年**是这样的，**2026 年**的话，架构创新能做的其实是越来越少了。"

2. K3 总览：三个维度的有效 Scaling

K3 的主要卖点是三个维度的 scaling。嘉宾认为**最重要的还是模型大小的 scaling**。

2.1 什么叫"有效"的 Scaling

"什么叫无效？无效的 scaling 我今天就可以给你：我晚上回去起一个 2.8T 的模型 init，我跑个 100B 的 token，然后我就可以放出来。这个显然是无效的。所以 **scaling 永远是要建立在有效的基础上。**"

2.2 三个维度

维度	K3 的数字	说明
总参数量	2.8T	在 2.8T scale 上做了一次有效的 scaling
激活参数量	~100B (104B)	发布时点比国内其他开源模型大很多
上下文长度	1M	原生支持，可解决更复杂的任务

2.3 K2 vs K3 的对比

	K2	K3
总参数	~1T	2.8T (约 2.8 倍)
激活参数	32.6B	~104B (超过 3 倍)
架构策略	基本沿用 DeepSeek-V3 结构，主要发力在 scaling 本身和非架构创新	引入大量新的、创新的设计元素
Scaling Efficiency	基准	K3 是 K2 的 2.5 倍

第一性原理：更大的模型才有更大的智能上限。如果只看 benchmark，你有无数种方式去达到一个对内或对外满意的结果；但从模型能力本质上，**模型参数大小还是最有效的方式**，而 Kimi K3 是相当早期、也相当成功地达到了这样的量级。

2.4 为什么 K2 到 K3 变化这么大

嘉宾的判断：这是一个**战略问题，不是技术问题**。

"K2 在那个时间点他们也可以不一样，只不过他们当时主要的重心是**把模型跑出来**。K2 当时主要考虑的问题是：怎么把 Muon 训出来、怎么有效地把模型 scale 上去。当时 K2 是 one training，比 DeepSeek-V3 还要大一些，在那个时间点已经是相当大的模型了。所以他们当时解决的是——**先把 base model 整体的 pipeline 做稳**。在稳的基础上，K3 再去追求一些更精益求精的优化。"

"（架构团队/预训练团队）内部其实一直在做，只不过看你什么时机把它亮出来。研究工作是更长期的，要看放在哪个模型里跟它一起结合。"

3. 模型架构（本次分享的重头戏）

3.1 线性注意力发展史 → Kimi Delta Attention (KDA)

为什么必须讲历史：

"如果你直接读 Kimi Delta Attention 的公式，你会发现它相当复杂，比经典 Attention 要复杂很多。每一项背后都有一个历史性的因素。所以想了解 KDA 最后是怎么设计出来的，还是有必要去探索线性注意力经过了怎样的发展阶段。"

3.1.1 演进路线图

最原始 Linear Attention

$$S_t = S_{t-1} + k_t v_t^T \quad (\text{位置无关, 最简单})$$

▼ 加入衰减项 (位置相关)

RetNet (2023)

$$S_t = \alpha \cdot S_{t-1} + k_t v_t^T \quad \begin{array}{l} + \text{ chunkwise recurrent 计算范式} \\ \alpha = \text{位置无关的固定衰减} \end{array}$$

▼ 衰减从"位置无关"→"位置有关"

Mamba / Mamba2

$$S_t = \alpha_t \cdot S_{t-1} + k_t v_t^T \quad \alpha_t \text{ 随 token 变化}$$

▼ 提高"上下文容量" (Delta Rule)

DeltaNet

(概念来自更早的工作; Songlin Yang 解决了它的并行化)
在相同 KV cache 大小下, 从算法上提高线性注意力的上下文容量
△ 但当时没有位置衰减, perplexity / benchmark 结果不够好

▼ 衰减 + Delta Rule 结合

Gated DeltaNet

$$S_t = \alpha_t (I - \beta_t k_t k_t^T) S_{t-1} + \beta_t k_t v_t^T \quad (\text{示意})$$

α_t 是 **标量** 衰减 (整个 head 共用一个衰减系数)

▼ 衰减从标量 → channel-wise

Kimi Delta Attention (KDA) ← Kimi Linear / K3 采用

α_t 变成 **channel-wise / fine-grained** 衰减
+ Lower-bounded decay (为 kernel 做的 co-design)

3.1.2 逐项拆解 (公式怎么"长出来"的)

以 Gated DeltaNet 的递推式为参照：

- 如果把 S_{t-1} 前面的系数完全消掉 → 最经典的 Linear Attention;
- 如果只保留一个衰减系数 → RetNet / Mamba (对上一时刻的历史状态做一个衰减);
- 再引入 **gated delta rule**, 写成 DV 形式则额外多出一项 → Gated DeltaNet;
- KDA 在 Gated DeltaNet 基础上, 只改衰减项。

复杂度就是这样一层一层叠加上去的。

3.1.3 KDA 的核心改动: 标量衰减 → channel-wise 衰减

线性注意力是分 head 的, 但写公式时一般只考虑单 head 情况。

Gated DeltaNet 及之前		KDA
衰减项形式	标量（整个 head 共用一个衰减系数）	channel-wise （每个 channel 衰减系数不同）
写 kernel	容易，一个 tile 里只有一个变量	麻烦很多
表达能力	较弱	严格更强（channel-wise decay 可以退化为均匀 decay）

结论：从公式上讲 **channel-wise** 是严格更好的，因为它一定可以退化到均匀 decay。严格更强的模型能力带来更好的效果。代价是 kernel 写起来更麻烦——不能再用“整个 tile 一个变量”的便捷方式去利用 TensorCore。

3.1.4 Lower-Bounded Decay：一次典型的 algorithm-infra co-design

这是本节最精彩的部分。为什么要给 decay 加下界？

- 表面理由（算法层面）：我们希望它不要 decay 得太快。
- 真实理由（paper 本身给的）：为了和 kernel 做 co-design，即数值精度问题。

推导链条：

① 为了高效计算 **fine-grained decay**，需要用“绝对位置表示相对位置”的技巧。

这和 **RoPE** 的思想完全一致——RoPE 在 infra 上特别方便的地方，就是它可以通过绝对位置变换来表示相对位置变换。

举个最简单的类比： $\boxed{-2}$ 可以表示为 $\boxed{3 - 5}$ 。

② 具体做法：对 Q 和 K 分别做 decay 的正/倒数变换。

比如衰减项是 $\boxed{\alpha^2}$ （可能是第 0 个 token 对第 2 个 token，也可能是第 100 个对第 102 个）。用绝对位置表示时：

- 让 Q 先做一个较大的衰减（比如 $\boxed{\alpha^3}$ ）；
- 让 K 做一个反向的提升（比如 $\boxed{\alpha^{-5}}$ ）；
- 通过乘法结合律，得到和原来递归等价的形式。

③ 问题来了：这会导致 QK 的某一边要乘/除一个很小的数。

- 乘一个很小的数问题不大——无非就是 decay 到 0（虽然我们也不希望 decay 到 0）；
- 但从数值精度上讲，我们希望它控制在 **BF16 的动态范围内**，否则“用绝对位置表示相对位置”这个技巧就失效了。

④ 解法：把衰减在一个固定 **chunk** 内部的累积量，从数学上控制住。

- Kimi Linear 用的是 **16-token tile**；
- 目标：这 16 个 token range 内的衰减不超出 BF16 的动态范围；
- BF16 的精度范围是已知的 → 反解出在 16 个 token 区间内最多能衰减到多大的量；

- **KDA** 就选了这样一个下界，并可以证明在 16-token tile 内衰减项不超出 BF16 dynamic range。

一句话总结：Lower-bounded decay 不是为了模型能力，而是**通过算法层面的限制，保证 kernel 内部不出数值问题，从而让 kernelize 更通畅**。不加这个限制 kernel 也能写，但会低效很多。

3.1.5 KDA 的 TL;DR

1. KDA 把 Gated DeltaNet 的计算能力扩展了（标量衰减 → channel-wise 衰减）；
2. 为了弥补扩展带来的**额外 infra 麻烦**，在模型的表达范围内**采取了一些限制**（lower-bounded decay），让它满足数值上可 normalize 的方式；
3. 最后得到 K3 采用的形式。

3.2 Gated MLA

顾名思义：**MLA + Gated Attention** 两块的结合。

3.2.1 MLA 部分

- MLA 最早是 **DeepSeek-V2** 开始用的，主要目的是**减少 KV cache 开销**；
- 后来被 **DeepSeek-V3** 和 **Kimi K2** 采用；
- **DeepSeek-V4** 没有用 MLA。

K3 为什么继续用 MLA：Kimi 的同学认为，MLA 在全注意力这个 scope 内部至少是**可以接受的形式**，或者说没有**必要去大动**，所以 K3 沿用了和 K2 一样的形式。

嘉宾的补充判断（从组织管理角度）：

"K2 当时主要是 follow DeepSeek，他们当时的选择是先不做架构创新，先把结果跑出来。从 K2 到 K3，从工程或组织管理上讲，**能跑的东西就不要动**。如果 MLA 不是一个特别大的问题，那可能就暂时不会动。但未来也不一定，我觉得这是一个比较暂态的状态。"

3.2.2 Gated Attention 部分

- 最早是千问（Qwen）团队开始用的；
- 从 Qwen 的论文标题可以看出它当时解决的问题：**non-linearity、sparsity、attention sink-free** 等；
- 但大家最 **buy-in** 的一个点是：**训练稳定性**。

"这个训练稳定性是被广泛验证过的。对一个 pretrain 的 run 而言，我们还是希望它越稳越好。从模型能力上**如果不是最优的也没关系**——因为模型能力最主要的因素是参数大小，所以好一点差一点都不如稳定性重要。如果稳定性不好，最坏的情况是整个 run 训崩了，前面所有 checkpoint 都废掉了——这对任何团队而言都是不可接受的。"

采用情况：Qwen3 应该还没有用，**Qwen-Next、Qwen 3.5 / 3.6** 应该都采用了。虽然会带来一定额外计算开销，但对稳定性好处特别大。

3.2.3 为什么可以结合

Gated Attention 与 GQA / MLA 是正交的设计——任何 attention 变体都可以用。所以 K3 把 Gated Attention 的稳定性手段引入 MLA:

- 不破坏 MLA 本身的推理性质;
- 提升训练稳定性。

3.2.4 【访谈】MLA 的未来会是共识吗?

嘉宾的技术判断:

"MLA 本身就是 MQA 的一个等价形式, 所以 MLA 本质上并没有带来新的东西, 它只是一个更好的 **trade-off**。大家没有发现的一点是: 如果想把 MLA 用好, 想达到相同的计算效果, 它其实会增加更多的 Tensor 计算去弥补这方面的损失。我个人认为 **MLA** 主要拿到的收益, 其实可以被更好的 **GQA** 参数设计基本拿到绝大多数。"

DeepSeek-V4 为什么不用 MLA:

"因为 V4 后面主要推 sparse。Sparse 跟 MLA 不是 100% 兼容的, 尤其是在 prefill 阶段。而且他们其实是用了 MLA 的等价 MQA 形式——因为从数学上讲 **MLA 本质上就是一个大号的 MQA**。所以 DeepSeek-V4 用的是一个大号的 MQA, 本质上区别不是很大。"

罗福莉的观点 (主持人转述): MLA 对于 Chat 来说是优秀的模型结构, 但不那么适合 Agent 范式。MLA 设计之初是为了在当时的 H 系列芯片上达到很好的访存/计算比例, 既不浪费算力又打破访存瓶颈; 但发现计算剩余实在太多, 所以 MTP 能更有效地利用起来。

嘉宾的回应:

"这是一个比较 reasonable 的点。MLA 在推理阶段的计算其实是大大浪费掉的——它的计算远大于它实际的建模能力。罗福莉的主要讨论点是: 这种计算模式跟 MTP 的收益不是正交的, 甚至可能是互相缓冲的。因为 **MTP 本质上就是用更多 token 同时去计算, 在 memory-bound 条件下往 compute-bound 方向推**。所以对于 MLA 而言, MTP 的加速比会大大减小。"

"我觉得 **MLA** 从来也没有成为共识, 它只是一个选择。"

3.3 Attention Residual: 从 ResNet 到 DenseNet 到 HyperConnection

论文 2.2 节。嘉宾认为, 如果 follow 这条研究脉络, **HyperConnection** 才是最好的那篇工作。

3.3.1 源头: ResNet 与 Pre/Post-LayerNorm

Attention Residual 本质上和 ResNet 有深刻联系。ResNet 解决的是: 模型能力在参数增长的情况下至少不退化; 无论从稳定性还是能力上, 它相比之前的工作都有本质性提升。

一个大家没注意到的细节:

ResNet 有两个版本——一个 **CVPR** 版本, 一个 **ICCV** 版本。其实在 ResNet 那个时代, 何恺明就已经讨论过 pre-LayerNorm 和 post-LayerNorm 与训练稳定性的关系了。

这条线也可以追溯到 BERT 时代:

- BERT 时代大家更常用 **post-LayerNorm**, 觉得结果稍微好一些;

- 但模型逐渐增大后，**post-LN** 出现梯度消失 / 训练不稳定的问题；
- 所以后面大家逐渐改用 **pre-LayerNorm**。

3.3.2 苏剑林的关键观点

Attention Residual 绝对不是"pre-LN 换到另一种 LayerNorm 摆放方式"的区别——那样风险很大，因为一旦触及 post-LN 之前的问题，大的 run 大家肯定不敢上。

所以 attention residual 必须强调：**它一定是 pre-LayerNorm 的一个超集，至少可以退化到 pre-LayerNorm，是严格比 pre-LN 更强的结果。**

3.3.3 HyperConnection（字节 Seed 团队）

嘉宾认为**这是这条线上最有意思的工作**，无论是 DeepSeek 后来做的改进还是 attention residual，本质上都是从这篇来的。

它解决的问题：在 pre-LayerNorm 的基础上，**如何增加模型残差的容量**，从而提升模型的表达能力。

HyperConnection 的一句话总结（嘉宾的提炼）：

"我们在 residual 这个分支上，**用比模型 hidden state 更大的容量**，来表示模型在推理深度上的状态。有更大的容量，一定有更好的效果。"

为什么没出圈：

"我跟 HyperConnection 的一作交流过，我说这个工作从技术上很好，但**最大问题是 paper 写得太抽象了，不太容易读**。思想上相当简洁，但呈现形式相对复杂。所以当时没有出圈，只是做架构或技术层面的同学才会去了解。"

"后来 **MHC 比 HC 本身火了很多**，但我觉得最重要的原因还是 **MHC 是 DeepSeek 提的**，我觉得可能没有别的原因。"

3.3.4 为什么这类工作大家愿意用：推理免费

关键判断：

HyperConnection / Attention Residual 这类工作的一个好处是——**对推理来说基本是免费的**。在 inference 阶段几乎不会增加任何推理开销，就可以拿到更好的模型结果。

反面：

"一旦涉及到通过更慢的实现、更慢的架构设计来取得更好表现，就带来一个自然的 trade-off：**我为什么不去扩更大的 size？这样还简单一些**。所以一切通过更慢的模型实现去提升效果的方法，都会被 fully question；对于保守的、或者不是自己提的方法，大家甚至就不会愿意用。"

"而对于推理阶段几乎免费的设计，大家上的时候没有什么压力。"

3.3.5 绕不过的 DenseNet（黄高）

DenseNet 是 ResNet 之后提出的连接方式，思想上和 attention residual 相当密切：

Residual DenseNet		
连接范围	层和层之间	深层去看所有浅层的状态，再聚合
聚合方式	加法	一个大的 Linear （那个时代没有 Attention，也没有过度重视 Attention 的表达能力）
速度	快	不快，中间连接带来大量额外计算

Attention 时代的改进：把 DenseNet 里比较 heavy 的部件换成更 lightweight 的部件 → 诞生了 **Attention Residual**。

3.3.6 Attention Residual 的两个效率要点

(1) Attention 本身的计算比较 lightweight（相比 DenseNet 的大 Linear）。

(2) Block Attention：解耦掉模型深度

问题：如果用全注意力，上层会和底下所有层交互——模型越深，这块的 **overhead** 越 hold 不住。

解法：用 **block attention** 在这里做解耦。

代价与结论：

Block attention residual 因为解耦了深度，严格来说表达能力肯定不如 full attention residual 好。但作者 claim 的点是：它并没有损失太多能力，而且相对 **baseline** 有显著提升。

K3 在这块应该没有做太多特殊处理，主要沿用上一篇技术 paper 里讲的内容。

3.3.7 【访谈】这几个里哪个创新性最强？

"我更喜欢 **DenseNet** 和 **HyperConnection**。

DenseNet 是相当早的工作。在那个时代，本质上 attention residual 考虑的问题 DenseNet 都考虑到了，只不过当时没有 attention 而已。

HyperConnection 相当于在 Transformer 时代重新把这个东西带了回来，让大家在连接方式上重新讨论：如何有一个更强的连接方式来提升性能。后面的工作当然会有一些 co-design、一些稳定性的改进，但大的框架其实是之前的工作已经定好的。"

3.4 Latent MoE

由英伟达团队在（2026）年初提出，K3 follow 了这个结构。

3.4.1 背景：MoE 的最大挑战是 All-to-All

训练 MoE 无论从稳定性、训练方式还是 infra 上都是很大的挑战。最大的挑战是 MoE 的 **all-to-all dispatch**——在 EP（Expert Parallel）情况下 **overhead** 相当大。

为了解决这个问题，业界不得不引入 **DeepEP** 这样高度优化的通信库来降低 all-to-all 开销。

重要观察：不同的 MoE 设计方式不仅影响模型架构本身，甚至影响底层 **infra** 的设计选型。

3.4.2 Latent MoE 的核心思想

通信量的构成：all-to-all dispatch 要把 hidden state 分发到各个专家卡上，通信量随两个因素并行增长：

- 1. Expert 激活的数量
- 2. 模型 hidden state 的大小

Latent MoE 的做法：把 dispatch 的 hidden state 减小（比如减小 2 倍、4 倍），从而减少通信量。

问题：直接减小 hidden dimension，后面 MoE 的参数量会随之减少。

两种弥补方式：

- 1. **提升 FFN intermediate dimension**——通过提升 FFN 本身的参数量补回来；
- 2. **进一步把模型拆碎**——用更多的 expert 和更多的激活来弥补参数量的减少。

结论（英伟达 paper 中结果更好）：

一个恰当设计的 **Latent MoE configuration**，可以完全保持 **standard MoE** 的结果，甚至还能好一点。也就是说，可以在模型能力完全相等甚至更好的前提下，减小通信开销。

3.4.3 为什么这是"接近 free lunch"

关键洞察：通信开销在训练和推理中的性质是不一样的。

	训练	推理 (latency)
优化目标	throughput	latency
通信能否掩盖	可以用 overlap 手段弥补掉	尽管可以用手段掩盖，但时间避免不了，因为它整体在 critical path 上

所以从 efficiency 上讲，**推理的收益比训练的收益要大**（训练也有一定收益）。既然 Latent MoE 不是一个 trade-off（不需要牺牲表达能力来降通信），它其实是一个 **free lunch** 或类似 **free lunch** 的结构。

3.4.4 细节改进：两个矩阵连乘必须加 Normalization

这是一条可以直接记下来的通用经验法则。

问题陈述：两个矩阵连乘是非常基本的单元。从表达能力上讲它是冗余的（两个矩阵连乘严格可以合并成一个）。但从优化性质上讲并不是这样——两个矩阵连乘经常会出现模型训练不稳定的问题。

通用法则：

在模型架构设计中，如果你不得不把两个矩阵连乘拆开来写、而不是合并到一起，**中间一定要加一个类似 normalization 的手段**，把中间的 hidden state 控制住，从而提升整体训练稳定性。

两个应用实例：

1. **MLA**：本质上是先做一个 low-rank projection 再打回来，也是两个 linear 连乘 → 中间用 normalization 控制 hidden state；
2. **Latent MoE**：先用 linear projection 把 hidden state 压到低维，然后在此基础上做后续计算。而 FFN 的第一个计算也是 linear projection → 出现两个 **linear projection 连乘** → 需要一个 **RMSNorm** 来控制。

3.5 C2GLU：给 MLP 中间激活加数学上界

3.5.1 问题：FFN 中间激活容易爆炸

对大模型训练来说，FFN 这块的 MLP 中间的 **hidden state** 特别容易出现 **activation explosion**。

两个诱因：

1. 跟优化器有关；
2. 跟模型精度有关——精度更低的情况下，中间 **hidden state** 更容易出现 **outlier**。

3.5.2 演进：Hard Clip → Soft Clip → C2GLU

① gpt-oss 的做法：Hard Clip

最简单粗暴：把 SwiGLU 中没有 bound 的那块直接通过一个 clip 控住（比如设 clip 值为 5 或 10）。数学上比较严格地把上限控制住。

② 从 Hard Clip 到 Soft Clip: tanh

"clip 是一个比较粗暴的状态。你如果把 hard clip 变成 soft clip，就会比较自然地得到 **tanh** 这个算子——因为 tanh somehow 就是一个 soft clip。指定一个下界、一个上界，它可以用比较平滑的方式逼近你希望的上界。"

嘉宾的态度：

"可以理解为 clip 的一个升级版。但这个升级版具体在模型表达能力上有多大影响，是另一回事，肯定需要具体实验去证明。但 whatever，**soft clip** 还是一个比较安全的做法。"

③ C2GLU 的完整构造

从 SwiGLU 出发拆解：

- SwiGLU = 一个 Linear 的结果 × 一个 Swish 的结果；
- Swish 又可以拆成 **Sigmoid × Linear Projection**（这里是共享 weight 的状态）。

C2GLU 的做法：把所有中间可能出现 unbounded 的环节，都通过 **tanh** 的上下界放缩 **bound** 住：

- 第一步：把比较自由的 Linear Projection 改成 **tanh-bounded** 的状态；

- 第二步：对 Swish 拆出来的 Linear Projection 同样处理。

结果：整个 MLP 中间的激活获得了一个严格的数学上界，用于保持训练稳定性。

3.5.3 【延伸讨论】Muon 与 Outlier：与苏剑林的（预判的）分歧

嘉宾的观察：

"如果用 **Muon** 的话，它其实更容易出现 outlier——你控制中间的激活值，总不是一个比较错的现象。"

嘉宾**预判**苏剑林会反对：

"苏剑林肯定会说：对于任何优化器来说，它都一定会出现 outlier，因为你从模型架构上是控制不了的。所以如果想严格控制，一定是直接从模型架构下手。"
(主持人问"苏剑林跟你观点不一样?") "这个他没有说，但我猜到他会这么说。"

依据是 **K2** 时期 **MuonClip** 的类似讨论：

- 因为用了 Muon 优化器，**QK logits** 更容易爆炸，所以 Moonlight 采取了一种对 QK 的特化处理方式来压制 outlier；
- 当时大家问：为什么之前用 **Adam** 训练不需要加这个东西？之前 QK logits 都很稳；
- 苏剑林的回复：严格来说 **Adam** 也不是完全稳定，DeepSeek-V3 可能 somehow 也有一些不稳定现象，只是没有影响最终结果。在 Muon 训练里出现脱落（outlier）现象是**严格来说容易出现的，只不过是早晚问题**。所以为了严格限制模型行为，从模型架构本身去做限制是一个更本质的方案。

贯穿的方法论：为什么不直接在 architecture 上做改变，把 bound 直接限制住，而不需要去纠结更复杂的优化细节？

3.6 优化器：Muon 一脉的稳定性工程

K3 在优化器上相比 K2 没有做太多改变。

Muon 的演进历程：

阶段	做了什么
早期 Muon	更粗放，连 weight decay 都没有加，控制不住中间的 activation outlier
Moonlight	率先把 weight decay 引入到比较大规模的模型训练里，使更长的 run 能够保持稳定
Kimi K2	在 weight decay 基础上，额外加了 QK-Clip 的放缩方式

这两个加起来，是 Kimi 团队在优化器上一个比较完整的工作。

为什么 **Kimi** 用 **QK-Clip** 而不是 **QK-Norm**：

"如果你用 MLA，你就没有办法去加 **QK-Norm**（QK-Norm 是严格限制 QK 边界的）。所以为了适配 MLA 和 Muon 的结合，K2 引入了 QK-Clip 这样的操作。"

其他家的做法：

"DeepSeek-V4 包括 StepFun 那些，他们应该都用了 Muon。在 Muon 基础上，因为他们都用的 GQA、没有用 MLA，所以直接用 **QK-Norm** 本质上是一个更简单的办法，去控制 Muon 带来的额外 outlier。"

Outlier 出现的两个位置与对应手段：

位置	手段
QK 那块 （更容易出现）	clip 或 normalization 限制住
MLP 那块 （也一定会出现）	更 lightweight 的手段—— Kimi 现在是通过设计激活函数（ C2GLU ）把它 bound 住

3.7 Quantile Balancing: MoE 负载均衡的"更 principled"解法

没有发论文，是苏剑林在自己的博客里讲的。K3 paper 中补充了博客未详述的工程实现。

3.7.1 前身: Loss-Free Balancing

做法：只通过一个 **batch**（而不是整体的 loss）来控制 MoE expert 的负载均衡。核心是维护一个 bias：某个专家被选得多，就把 bias 往下砍一点；被选得少，就往上加一点。

当时大家觉得奇怪的点：这个 batch 变量直接决定了每个 token 选哪些 expert，但它的更新方式 **somehow** 是 **ad hoc** 的——采取了一个启发式的更新方案。

两个问题：

- 1. 没有一个严格的数学收敛标准；
- 2. 容易和主模型的更新耦合得不够好（"给大家一个感觉是可能没有那么耦合"）。

Loss-Free 最主要的遗留问题：

使用 Loss-Free 时，大家往往会把模型最底下的一层或三层的 **MoE** 去掉（改成 **dense**）。原因是：**Loss-Free** 这套控制方案对底层模型 **work** 得不是很好——正因为它没有做整体的 coordinate，所以只能采取别的方式来弥补大规模训练中遇到的问题。

重要澄清：前几层用 **dense** 并不是 **ground truth** 上一定要这么做的方式，而是在 Loss-Free 这个 context 下不得不采用的方式。如果不用 Loss-Free、直接用辅助 loss 方案（比如千问 MoE 的做法），**第一层**也不会有问题。

3.7.2 Quantile Balancing 的核心

思路：能不能直接通过线性回归的方式，直接推导出这个 **bias** 应该长什么样？

答案：可以推出来，而且推出来的具体方式跟之前的 **Expert Choice** 有一定关系。

最终形式：经过一系列推导，最后得到一个**异步的形式**——对一个 step 内部的整体激活求一个 bias，这个 bias 本身就是让模型负载均衡的方案。

为什么是异步的（下一步才用）：

苏剑林博客中提到，**为了避免信息泄露，不能让任意一个 token 获得其他 token 的信息**。所以这个 bias 不会在当前 step 使用，而是在下一个 step 使用。

3.7.3 两个优点

优点一：不需要调参

Loss-Free 有一个类似 learning rate 的更新超参，QB 少了这个参数，整体看起来更 principled。

优点二：负载均衡能力更优

博客中贴出的图显示：对于第一层，QB 就可以直接把第一层做到 balance，不需要再把前几层改成 dense 来规避问题。

兼容性：博客中也提到，如果本来 Loss-Free 就比较 work 的情况下，QB 也是比较 work 的。而且可以证明：如果 QB 不采用直接均衡、而采用梯度更新的方案，它可以和 Loss-Free 做到一个相当等价的数学形式。

3.7.4 K3 论文补充的工程实现（博客未提及）

问题：现在大模型训练一个 batch 特别大——4M token 可能都算小，可能到几十 M 的量级。QB 需要在几十 M 的 token 之间恰好取到某个比例的分位数。如果要精确计算，需要把所有 token 的激活情况都存下来——这相当 heavy，甚至在工程上是不可能的。

博客中的方案：每个 GPU 做一个 local 的分位数计算，然后把不同 GPU（或者说把大 batch 切碎后的小 batch）的分位数结果做一个 pooling。

K3 paper 的不同方案（嘉宾注明"也有可能是我理解错了"）：

不是按 token 切 chunk，而是对模型的值域切 chunk（分桶）。

比如对 sigmoid 的输出，它一定在 0~1 之间（或 -1~1）。那么就在这个区间内切成 N 个桶，在桶里做 Histogram 统计。

这样做的三个好处：

1. 常数级存储——存储大小是常量，与 token 数无关；
2. 计算代价比朴素实现高效很多；
3. 可以比较好地扩展到大规模 DP/EP 的分布式架构下。

总结：naive 的实现基本不可行，必须用工程上可行的方式去把这个比较精确的解得出来。这是 QB 一个"比较必要的实现方式"。

3.8 Vision Encoder：为什么 from scratch

论文 2.4 节。核心讨论点：是否有必要用已有的 vision encoder 去做初始化？

用已有 **vision encoder**（如 **SigLIP2**）的好处：收敛更快。

K3 发现的问题：

如果直接用 **SigLIP init**，模型的不稳定性会比直接 **native train** 来得大。

关于"不稳定性"的一个重要限定：

"也不一定是模型不稳定性，因为不稳定是一个比较相对、甚至不是严格被定义的东西。只是大家一般会去看 **gradient norm** 的分布，中间有一些东西是大家比较 concern 的：比如尖刺不多，甚至本身的 **norm** 大不大。"

K3 的两个 claim：

1. **From scratch** 训练可能会多用一些算力，但最终结果不会有损——经过恰当的训练，不会带来更差的结果，甚至可能差不多或更好；
2. 原生训练带来更好的兼容性，体现在 **gradient norm** 更加稳定。

嘉宾给出的一个可能解释（优化器兼容性）：

"**SigLIP** 是用 **Adam** 训练的。之前大家讨论过一个相关话题：用 **Adam** 训练的模型，是否能用 **Muon** 去 **finetune**？反过来用 **Muon** 训练的模型，是否可以用 **Adam** 去 **finetune**？

最后的结论就是：比较原生的方式肯定是最好的。用别的方式如果想达到一样好，可能就得做一些处理，或者说是有一些限制。

所以对 **Kimi** 而言，他们希望采用一个全 **Muon** 的优化方式，这样对模型整体的训练稳定性和兼容性是一个更优的状态。"

3.9 为什么 K3 不用 Sparse Attention

三种 attention：线性注意力、全注意力、**sparse attention**——可以三选一、三选二。K3 选了前两个。

3.9.1 Sparse Attention 在新硬件上的困境

已有的采用者：**DeepSeek-3.2**（**DSA**, **DeepSeek Sparse Attention**）、**GLM-5**（也用了 **DSA** 架构）。

核心问题：

DeepSeek Sparse 在 **decode** 里的加速其实特别特别小，原因在于——求 **index** 这一步相当昂贵。而且在 **Blackwell** 硬件上，越先进的硬件这个 **overhead** 越大。甚至在 **decode** 当中，**sparse attention** 方案相比 **full attention** 并不会带来显著加速。

3.9.2 解法：跨层复用 index

GLM 团队的 **Index Cache**：把"求解哪些 token 需要算 attention"这个计算操作在不同层之间共享（比如每 4 层、每 8 层共享一次）。只有这样才能有效减小 **sparse** 本身的开销；只有 **sparse attention** 本身开销足够小，才能带来可观的算法收益。

嘉宾团队的做法（更极端）：在 YOCO 架构上直接 leverage sparse attention——因为 YOCO 的 KV cache 是 once 的，所以直接把 **sparse index** 这一步也做成 **once**，把每层的 index 开销分摊到所有层上。这样开销特别小，才能吃到 sparse attention 本身的计算收益。

两条路的结论一致：必须把多层的 **sparse** 结果跨层复用。

3.9.3 K3 不用的两个原因

原因一：跨层复用与 Hybrid Attention 不兼容

跨层复用最简单的方式是相邻层去跨。但对于 hybrid attention（线性注意力和全注意力 interleave）来说，不同的 **full attention** 层并不是挨着的——每个 full attention 中间隔了很多个线性注意力层。在隔了很多层的情况下，这种 **index share** 是否有效、加速比是否足够高，是相当值得怀疑的。

所以“想拿到实际加速比”对 hybrid attention 来说是一件 **non-trivial** 的事情，需要做更多工作。

原因二：Sparse Attention 和 Pretrain 不兼容

Sparse attention 基本上没有办法进行 **from scratch** 训练。现在大家一般采用的都是 **post-train** 从 **full attention** 进行转化的方式。

不排除 K3 未来会把已有的 full attention 部分转化成 sparse attention。当然，如果他们用的是 **MLA**，这个转化会更难一些。

结论：这其实是“留一个接口，之后再做也行”的状态。

3.9.4 【访谈】架构上主要的 trade-off 是什么？

“**Hybrid attention** 主要的 **trade-off** 就是调节线性注意力和全注意力的比例。线性注意力比例越大，加速比越高。但如果你想达到一个偏无损的加速状态，**3:1** 就是实验上比较 work 的方案。Sparse 严格来说大概率不会带来更强的表达能力，所以它跟 hybrid 是另一条线。只不过如果你坚持不用 hybrid，那你可能就不得不采取一些别的方案了。”

4. 预训练

4.1 Learning Rate Schedule: K3 为什么放弃 WSD

这是 K3 与“几乎所有团队”都不一样的一个 setting。

4.1.1 WSD 的由来 (MiniCPM)

WSD = Warmup-Stable-Decay，由 MiniCPM 提出。

它的出发点：如何把 **data schedule** 和 **learning rate schedule** 结合起来。

推理链条：

1. 观察：WSD 和 cosine decay 可以取得相当接近的结果。也就是说，“从高学习率到低学习率中间怎么调配”对最终结果影响没那么大。
2. 既然中间的 **decay** 方式对结果影响不大，那我们就可以拿它跟 data pipeline 做联动。

3. 另一个观察：在 decay 阶段，模型 loss 的下降速度比"用大学习率稳步跑"要快很多。所以大家认为 **decay 阶段是模型学习最快的阶段**。
4. **MiniCPM 的策略**：既然 decay 阶段学习最快，那为什么不把更多、更好的数据放在 **decay 阶段训练**？这样可能比"data 均匀分布 + learning rate 均匀分布"取得更好效果。

嘉宾对这篇工作的评价：

"我觉得挺有意思的。一方面它讨论了一个 non-trivial 的问题：**为什么我们要采取不同的 learning rate decay 方式？decay 的不同阶段具体承担什么样的 role？**然后利用不同 decay 阶段的行为，跟 data 的具体策略结合，最后取得更好的结果。"

4.1.2 WSD 的第二个好处：Token 数可以自由决定

因为 WSD 前面的 learning rate 是不变化的，既然不变化，它就跟你整体要训练的 **token 量** 无关。

所以可以更自由地在训练过程中选取最终 config：

- 原计划跑 10T token，跑到中间训练策略变了、或者公司策略变了、或者要更早/更晚发版；
- WSD 允许你在中间自由切换，而不用在一开始就把要训的 **token 量** 定死。

4.1.3 K3 指出的问题：这个好处被高估了

虽然可以任意选择实际跑的 **token 数**，但你实际的 **learning rate** 是和要跑的 **token 数** 有关系的。

举例：假设 **6e-4** 作为一个比较稳定的 schedule——

- 你跑 **10T**，6e-4 可能是最优的；
- 你跑 **20T**，可能 **3e-4** 才是最优的。

所以从"最优"的角度考虑，任意扩展 **token 数** 可能并没有大家想的那么有效。

而且，"任意选取最终训练 token 量"这件事也不一定只有 **WSD** 能做到，用别的方式也可以做到。

4.1.4 K3 的选择：回到 Cosine Decay

核心理由：超参更好调。

Schedule	需要调的变量
Cosine Decay	① 跑多少 token ② maximum learning rate → 2 个
WSD	① 跑多少 token ② maximum learning rate ③ decay 的比例 → 3 个

结论：从调参角度，cosine decay 是一个更容易调参的方案。K3 因为发现 cosine decay 更容易找到好的 hyperparameter setting，最后采取了这个"比较经典"的 learning rate decay 方式——这在大家都用 **WSD** 的情况下，是一个不太一样的 **setting**。

4.2 Scaling Efficiency: K3 是 K2 的 2.5 倍

	K2	K3
总参数	~1T	2.8T（不到 3 倍）
激活参数	32.6B	~104B（超过 3 倍）
Scaling Efficiency	1×	2.5×

关于这个数字的重要澄清：

有人问：K2 到 K3 的 scaling behavior 对比，是不是固定 data 的？
"我记得他们的回答是应该不是，paper 里也应该没有直接写是不是。当然也可以理解——K3 和 K2 用的 data 肯定不可能是一样的，所以大概率有一个不一样的 data recipe。"

所以这个 2.5× 是一个大的综合结果：综合了 ① 模型架构 ② training recipe ③ 数据策略，叠加之后的总结果。

嘉宾也提到，对于简单 dense model，scaling efficiency 是一个经典讨论问题，之前 OpenAI 的 scaling law 以及一些较新的 scaling law 工作都讨论过。

4.3 长上下文与位置编码：NoPE 的优雅解

K3 是原生支持 1M 长上下文的模型。这块最有意思的点是位置编码如何选取。

4.3.1 RoPE 在长文扩展时的麻烦

传统架构用 RoPE 做位置编码，但扩长文时需要调整 RoPE 的具体参数（如 base / theta），不调整的话直接扩长文效果很差。这是 RoPE 时代的经典做法。

如果你还是用全注意力（不引入线性注意力），这个策略不会有太大改变——你还是得针对不同长度 setting 去调节不同的 RoPE 参数。

4.3.2 混合模型带来的改变：Full Attention 可以用 NoPE

最早是 Cohere 团队（音译，存疑）的一个工作指出了这个因素。

结论很简单：

在 hybrid attention 的排布方式下，线性注意力已经引入了位置信息（无论你用 RoPE、RoPE + sliding window 还是 linear attention）。所以在 hybrid attention 的 setting 下，可以把 full attention 从 RoPE 改成 NoPE（去掉位置编码）。

两个好处：

- 1. 带来更好的模型表现；
- 2. 长上下文更容易扩展——NoPE 在模型扩长时不需要调整任何模型架构参数就能达到长上下文效果。

为什么说它"优雅":

"模型架构的参数选择当然应该跟具体长度没有关系，也没有任何理论支持说我们为什么要在不同长度 **recipe** 下采取不一样的参数。这种架构有效地避免了这一点。"

嘉宾的个人经历:

"我比较早看到这篇，第二天就去复现了一下，因为我当时看到就觉得特别有道理。复现了很 work。我当时就觉得这肯定是混合注意力里比较好的一个方式。我在会议里还审到了这篇文章，当时我直接给了一个 **strong accept**——我这几年可能都没给过几个 strong accept，因为我觉得这确实是一个很有效、也很优雅的方案。"

4.3.3 【重要澄清】关于 RoPE 与长文能力的常见误解

RoPE 本质上带来的是 recency bias。
RoPE work 最有效的点是：它对 **short context** 的建模能力特别有效，但它对长文来说是没有帮助的，甚至是有负面影响的。
"某些舆论观点认为是 RoPE 引入了长文的表现，这其实是错误的。RoPE 并不带来任何长文能力，它甚至是损害长文能力的。"

所以 K3 把 RoPE 在长上下文的 **full attention** 这块直接下掉。下掉之后：

- 直接外推效果好很多；
- 用 RoPE 的话，扩长度需要调参数；调了参数如果不训练，结果其实很差；
- 用 NoPE 不用调参数，外推效果很好，长上下文效果没有任何问题，甚至还要更好。

这是在混合注意力 **context** 下一个相当优美的解决方案。

5. 后训练

5.1 低精度训练应该在什么阶段引入

各家策略对比：

模型	策略
DeepSeek-V3	一开始就原生 FP8 训练
DeepSeek-V4	直接用 W4A8 做延伸训练
Kimi K3	先用高精度训练，在 SFT 阶段再把低精度 QAT 引入进来

K3 这个选择的两个理由：

- ① 项目管理角度（抛开技术）

"大规模训练采取更高精度肯定是更保险的方案，因为它无论如何也不会错。如果用低精度训练在大规模 scaling 中带来了其他问题，这是一个相当不可控的因素，而且更大规模的训练是很难提前被充分、精确验证的。"

② 技术角度

"我们是否有必要一开始就用低精度训练？起码从我的实验而言，应该也是没有必要的。Kimi 可能也发现是没有必要的——也就是说，对于低精度推理而言，from scratch 一开始就引入并不会带来任何模型能力的提升。W8A8 你在什么时候引入，只要你过了一定量的训练，最后结果可能都差不多。所以在 SFT 阶段引入是一个更保险、在技术上也不会带来额外损失的方式。"

5.2 RL

Kimi 从很早以前就非常重视 RL，从 K1.5 到 K2.5 一直在做持续创新，K3 里也充分引入进来。这些都是比较成熟的方案了，嘉宾未展开。

5.3 On-Policy Distillation (OPD)

论文 4.1.3。最早由微软亚研院董力老师团队较早讨论，相关工作是 **MiniLM**。

5.3.1 蒸馏的两种形态

	黑盒蒸馏	白盒蒸馏
能拿到什么	只能拿到输入和输出（比如通过 API），拿不到模型内部状态和 logits	可以获取模型本身，任意拿到推理过程中的任何状态
现状	大家现在比较常采取的方式	需要有模型权限

核心问题：在白盒场景下如何做蒸馏，才能比黑盒蒸馏表现更好？

5.3.2 MiniLM 的"反其道而行之"

	传统方式 (Forward KD)	MiniLM / OPD (Reverse KL)
谁生成	Teacher 生成答案	Student 生成答案
怎么学	把生成的答案用 SFT / Forward KD 的方式引入 Student	Teacher 对 Student 的每一步做纠正
类比	老师讲课，你把老师讲课的结果学进来	你自己做题、自己写中间的推理过程，老师指出你哪里做得不对，告诉你怎么去做

这就是 **Reverse KL**。Thinking Machines Lab 后来给它起的名字叫 **On-Policy Distillation (OPD)**。

共识：从蒸馏角度，"不直接学习模型的答案，而是在 Student 自己的推理过程中让 teacher 模型逐步 verify / 纠正"是一个比较有效的方式。

5.3.3 OPD 最初的用途 vs 现在的用途

最初的 **motivation**：大模型蒸馏小模型。

从公司角度，你可以有一个 Pro 版本和一个 Flash 版本。Flash 版本可以用 OPD 蒸馏更大的模型，从而在这个"比较便宜的模型"档位获得更强的竞争力——比 **from scratch** 训练一个 **flash** 模型效果更好。

为什么现在不这么用了：

"如果想做蒸馏，你至少得有一个大模型，再去训练一个小模型。但现在大家都不是这个 **setting**——大家一般都是搞一个旗舰模型，用它作为 inference 服务。在这个场景下，就不存在'大到小蒸馏'的行为了。"

（另外，如果想蒸馏黑盒模型，这套方案完全不可用。）

现在的用法：自己蒸馏自己（**Multi-Teacher** 合版）

① 非技术层面的原因：简化 **post-train team** 的管理

"Post-train 合版是一个比较 heavy 的东西。**Pretrain** 只要把数据合起来一块干就完事了，但 post-train 一般会用更复杂、更先进的训练策略，直接去做合版会有一些难度。所以从项目管理策略上，**OPD** 可以把整个 **post-train team** 的管理变得简化。"

② 技术层面的原因：把"异构的 **reward**"转化为"同构的模型"

RL 训练时不同能力对应不同策略：

- 训数学 → verifiable reward;
- 有 preference 需求 → reward model;
- 想扩展别的能力 → 每个能力对于 **RL** 来说都有独特的 **reward**。

Post-train 一般大家都会专项突破。想把不同专项模型的不同 **reward** 直接拿进来一朝合会，比 **pretrain** 的难度要大（不是完全不可能，但很难）。

而用 **OPD** 就简单很多：我们可以把不同的 **reward model** / 不同的 **RL** 范式，简化成不同的模型。

关键洞察：对于 **RL** 策略或 data 来说，它是高度异构的状态；但对于模型来说，它是高度同构的状态——因为现在不会为了不同任务专门设计不同的模型架构。模型都长这样的话，再做一个 **multi-teacher** 的合版就容易很多。

行业现状：这不是 K3 首先用的，现在是一个 **widely accepted** 的做法。嘉宾印象里小米、GLM 等应该都是这么做的。

5.4 Draft Model / MTP：值得延伸的方向

嘉宾认为 **K3** 在这块目前优化得不是很完善，之后还会有更深层次的工作。

5.4.1 参照系：小米的 1000 TPS 方案

小米之前推出了一个能达到 **1000 TPS** 的方案，主要用了几个技术叠加：

1. **TileLang** 团队的算子——可以高度融合不同阶段，比传统 vLLM 推理方式强很多的推理效率（可达三四百 TPS）；
2. 在此基础上加 **TensorRT** 的结果；
3. 再加 **DeFlash**（投机推理）带来的额外加速比。

适用场景：

"这主要服务于愿意付出更高成本、但想拿到更强推理速度的这一波用户。如果你是整体 **throughput** 服务，收益会小很多；但如果你是小 **batch size**、大 **throughput**，这套方案会强很多。"

抛开 TensorRT 那方面的贡献，最后就落到了：如何做更好的 **MTP** / 更好的 **draft model**。

5.4.2 脉络一：EAGLE-3 / MTP —— 利用大模型的 hidden state

Speculative decoding 最初的问题：

刚提出时，大家用的是一个和大模型没有关系的小模型去 speculate 大模型。这带来一个问题：你无法充分利用大模型中间的 **hidden state**。

改进思路：

如果可以利用大模型中间的 **hidden state**，你就可以把小模型做得更小；或者在相同参数下，接受率做得更高，最后带来总的更强推理效率。

EAGLE-3 和 **MTP** 都是这个思想。（MTP 如果 from scratch 还有一些别的好处。）

5.4.3 脉络二：DeFlash —— 连接 AR LM 与 Diffusion LM

Diffusion Language Model 最初的 claim：

在单用户 / **batch size** 比较小的情况下，可以获得比传统 Language Model 快很多的推理效率。

Diffusion LM 之前的两个最大问题：

1. **From scratch** 训练效率不够高，整体结果很难和传统 AR Language Model 匹敌；
2. 只能打小 **batch size** 的 **throughput**。如果是多用户、追求总 **throughput** 的场景，Diffusion LM 并没有优势。

DeFlash 的做法：统一这两者——试图利用 Diffusion LM 更快的推理方式，进一步优化 MTP / draft model 这块的加速比，最后得到更优的单用户 **throughput**。

5.4.4 关键洞察：Draft Model 和 Pretrain 高度相关

嘉宾与 DeFlash 作者陈健交流后的一个重点：

"即使我们用 **DeFlash** 这种更先进的 **draft model** 策略，它跟预训练的关系仍然是很大的。对于 draft model 来说，它需要利用大模型的一定推理能力，在此基础上再去进一步扩大。

也就是说，**MTP** 需要 **pretrain** 阶段给它提供一个接口。这个接口对下游 draft model 的提升会特别有帮助，而这部分是需要 **在 pretrain 阶段就引入的**。"

Draft model 还需要特化的 fine-tuning:

比如可以用 **KL loss** 替代传统的 **next token prediction loss**，这样在原 model verify 的过程中可以对得更齐，进一步提升 draft model 的接收效率。

5.5 RL Task

比较 specific，不同任务有不同的具体策略。嘉宾建议[直接看原文](#)。

6. Infra

总体原则：现在模型架构的关键设计需要和 infra 做 co-design；同时模型架构本身也会反过来影响整个 **infra structure** 的设计。K3 和 DeepSeek 系列在 infra 上有明显的设计区别，这是本节最有意思的地方。

Infra 部分大致分三块：**Pretrain**（大规模分布式）、**RL**、**Inference**（RL 本身涉及大量 inference，所以两者有重叠）。

6.1 KDA 的 Kernel 与 Context Parallel

6.1.1 Kernel 层面

前面 3.1.4 已述：为了追求更高的 chunk 效率，需要对 KDA 的衰减系数做出限制，从而用更好的 kernel 设计形式带来更强的 kernel-level 效率。

6.1.2 Linear Attention 的 CP 为什么更复杂

	Full Attention / Sliding Window 的 CP	Linear Attention 的 CP
概念 难度	相当容易——经典算法如 Ring Attention、Zigzag Attention	更复杂
原因	计算模式足够简单，而且很多地方省不了：至少得把跨 GPU 的所有 KV cache 拿到一个 GPU 上做计算	需要重新思考 chunk 的层次

6.1.3 解法：两层嵌套的 Chunk

回到 **chunkwise recurrent** 这个大框架（对 linear attention 是必备算法）：

单机、不开 CP 的场景：

- 拆成比较小的 chunk（比如 16 tile）；
- **chunk** 之间用递归方式计算；
- **chunk** 内部用 **parallel** 方式计算；
- KDA 的 decay 限制主要就是为了解决 chunk 内部 parallel 计算的高效性。

Linear Attention 的一个关键数学性质：

chunk 本身可以任意拆分。不同的 chunk 大小——512 个 token 一个 chunk，或者 8K token 一个 chunk——在数学上是完全等价的。

△ **对比**：这和早年一些"拼接式"方案不同。比如有些工作尝试 chunk 之间用 Linear Attention、chunk 内部用 Full Attention——这种策略下 **chunk** 大小就跟具体的 **architecture design** 有关了。但纯 **linear** 的是无关的。

既然无关，**chunk** 就可以任意分层：

1M token 的超长序列

- 第一层 chunk：跨 GPU
8K 放一张卡，8K 放一张卡 ...
→ 以"大 chunk"为单元，不同 GPU 之间采取递归计算
- 第二层 chunk：机器内 / kernel level
在 8K 内部进一步拆分 (如 16-token tile)
→ chunk 内部用 parallel 计算

一句话总结：Linear Attention 的 CP = **双层 chunk**。第一层在不同机器之间做 chunk，第二层在机器内的 kernel level 做 chunk。这个方案之所以 **work**，本质来源于"**linear attention** 的 **chunk** 可以任意拆分"这个特性。这是与 full attention 很不一样的处理方式。

6.2 MoE 的 EP：动态冗余专家

论文 5.2.1。Kimi 团队基于 DeepEP 进一步延伸的 EP 计算方案。

6.2.1 传统 Dropleless EP 的两个问题

Dropleless EP = 对每个 token 而言，选哪个专家不受整体约束，想选哪个选哪个。汇总后会发现不同专家的负载不一样。

问题

后果

① 各 GPU 分到的 token 数 整体执行时间不一样 → 互相等待 → 浪费 GPU 算力
不同

② 各卡 token 数不确定 通信要额外多一个阶段：先告诉所有 GPU"你要接受多少 token"，再把 token 发过去

理论最优是什么：

如果完全为了 infra，最优方案是带 **drop** 的 EP——每个 token 受到全局影响，通过一系列 allocate 方式让每个 expert 收到完全均匀的 token，达到完全 balance。

但 **drop token** 大家现在都不会用，因为对模型能力的损失相当明显。

所以 **infra** 的策略是：努力把 **infra** 从非最优导向最优，同时保持模型能力不下降。

6.2.2 K3 的方案：动态 EP + 冗余专家

传统静态方案：比如 128 个专家、EP 8，每张卡放 16 个专家——静态分配。

K3 的动态方案：

1. 引入一些冗余专家 (**redundant experts**) ；
2. 可以从数学上证明：存在一个策略，只需要增加一个小比例的冗余专家，就可以保证各卡 **token** 数的完全对齐。

"我感觉像是一个比较经典的数据结构与算法的问题，这个是可以证明到的。"

3. 实现方式：**online planning**——提前规划哪些卡应该有哪些冗余专家，从而达到不同卡上 **token** 数的完全均衡。

6.2.3 这个方案的上限

注意：即使从卡的角度是均衡的，并不代表专家层面是均衡的。这个算法只保证每张卡的总 **token** 数一定，但不保证每个专家的 **token** 数一定（因为一张卡里有多个专家）。

如果还想保证每个专家都均衡，那就没有办法了，只能采取有损策略去优化。如果大家不希望有损，上限就卡到这里——可以在这两者中间找一些更优的处理方式。

这是 Kimi 团队一个比较有意思的创新。

6.3 Memory Efficient Training: 大规模 Offloading

① 软件工程层面的细粒度控制

可以更细粒度地控制每个部件应该采取 **recompute(activation checkpointing)** 还是 **offloading** 的策略。

② K3 的特别之处：大规模 offloading 到 CPU

把中间计算得到的结果 offload 到 CPU 上。这是一个 **non-trivial** 的东西：

理论上，想让 **offloading** 不拖慢训练是有条件的——你得保证 CPU 和 GPU 之间的通信满足某个不等式，这样才有"用计算完全 overlap 掉通信"的可能。

③ 关键手段：所有可 offload 的东西都用 FP8 存

这样可以省一倍。省一倍可能才能从一个边界跨到另一个边界（即从"不满足不等式"变成"满足不等式"）。

具体边界怎么算，跟具体的卡、具体的集群环境、以及具体的模型 config 都有关系。

6.4 通信 Overlap 与 PP: K3 与 DeepSeek 的路线差异

这是本节最有意思的对比。

6.4.1 DeepSeek-V3 的两个 non-trivial 做法

① 细粒度的 MoE 级 Overlap

MoE 中间有两个通信：**dispatch** 和 **combine**，overhead 都比较大（在 Blackwell 上情况又不太一样，但无论如何是很大的开销）。

DeepSeek-V3 的做法：

把上一个 **batch** 的 **MoE forward + backward** 和下一个 **batch** 的 **MoE forward** 融合起来，通过重排序计算的先后顺序，把所有大的 dispatch 和 combine 通信都藏起来。

难度：

"想达到这点需要特别精细、也特别复杂的软件工程优化。我们需要把所有的 **forward** 和 **backward** 计算从原子计算的层面就拆开，然后手工进行重新排布——我觉得这还是有难度的。"（MicroCall 里也讨论过类似的事情。）

② DualPipe

为了缓解 PP 的气泡和不同 activation 的不均匀，DeepSeek-V3 采取了 **DualPipe** 的 pipeline 排布方式。（后面还有 **DualPipe-V**，效果一样但逻辑上更简单。）

起码从公开写到的内容看，**Kimi K3** 没有采用这两个优化方式，而是用了别的方式。

6.4.2 K3 的通信隐藏：架构选型决定 Infra 简化

因果链：

K3 采用 Latent MoE
↓
通信开销极大降低
↓
在整个计算开销中占比大幅减小
↓
想 overlap 就变得容易很多
↓
不需要跨 batch overlap, 用 batch 内部更简单的手段就够了
↓
K3 的具体做法：把 MoE 的前向/后向通信，
与 shared expert 的计算 overlap 在一起

为什么用 **shared expert**：现在绝大多数 MoE 都有 shared expert，而且 shared expert 有一定的计算量。

为什么传统 MoE 做不到：

"如果你用传统 MoE，通信开销特别大，用 **shared expert** 去 overlap 就很难 overlap 掉。overlap 不掉的话，你就必须把另一个 batch 拿进来，这样才能完美地把通信都隐藏掉。"

总结：用不一样的 MoE 架构，带来了不一样的 **overlap** 策略。这个策略的额外好处是：推理的 **critical path latency** 是免费的。从训练架构上，因为通信量更小，可以采取更简单的策略来排布通信气泡。

一个容易被忽略的点：

即使通过更复杂的方法完美地把通信藏掉，它其实也不是完全无损的——即使是完美的通信/计算隐藏，计算也会相对地慢一些。

针对这个问题，**DeepEP** 也做了类似优化：用足够少的 **SM** 占用来获得更大的整体通信 throughput，从而让 overlap 跑得更快。

6.4.3 PP 的处理：不做对称，而是"转移不均衡"

问题：

- 用 DualPipe / DualPipe-V 时，不同 PP rank 之间相对是**比较对称**的状态；
- 用**单向 PP** 则不对称。

不对称的两种解法：

1. 让它变得对称（DeepSeek 的路线）；
2. 把不均衡 **transfer** 掉（Kimi 的路线）。

具体的不均衡在哪：

朴素的 PP 中，前面的 **PP rank** 进入得更早，积累的 **batch** 更多，**activation** 的压力就越大。

K3 的做法：

把 **memory** 占用比较大的 **PP rank** 的数据，直接存到 **memory** 占用比较小的 **PP rank** 上。
通过这种比较 **lightweight** 的方式，规避了更复杂的 **DualPipe** 设计。

6.5 Muon 的分布式切分

Adam vs Muon 在切分上的本质差异。

	Adam	Muon
优化方式	element-wise	需要全矩阵做 Newton 迭代
切分影响	相当于把所有参数摊平后再切，怎么切都不影响优化结果	有影响——在某个 rank 上需要把一个矩阵的所有参数都保存下来

所以这块需要做一些特殊处理，才能让 Muon 比较好地在 optimizer 这个维度切起来。

6.6 多模态 Infra

核心判断：纯 language model 的 infra 想优化好已经需要相当多 effort；但对于多模态，每引入一项多模态功能，**infra** 难度都是直线提升的。

6.6.1 Vision Encoder 需要单独开 CP

问题：Vision token 在进入大模型之前会被压缩。

- 输入到大模型时可能是 **8K token**；
- 但在压缩之前可能是 **32K token**；
- 而这个压缩在 **vision encoder** 内部是没有做的——所以 vision encoder 内部计算时序列更长。

结果：

即使语言模型主干不需要开 CP，**vision encoder** 可能就需要开 CP。这需要额外的 infra 处理。

6.6.2 PP 的不均衡：VL Native Training 特有的问题

问题：做 VL native training 时，**VL token** 只占一定比例。

- 对于某些卡，可能是**纯文本**状态；
- 对于另一张卡，可能是**VL 比例相当大**的状态。

这会造成两个层面的不均匀：

1. **DP** 层面不均匀；
2. **PP** 层面也不均匀——PP 首先要保证不同 stage 之间比较均匀，流水线才能打满。如果在第一个 **PP stage** 引入了 **vision encoder**，它的计算强度就会显著增强，破坏整体流水线打满的状态。

K3 的解法：

把 **vision encoder** 不放在 **PP** 的最开头，而是放在中间或尾巴上。

这样，在前面的 vision encoder 或比较浅层的 language model 计算过程中，中间的 **PP stage** 是闲置的——就把这部分闲置利用上，提前把 **vision encoder** 后面的一些步骤算出来，从而规避掉第一个 PP stage 带来的巨大气泡。

6.6.3 【访谈】论文写这么详细，复现难吗？

主持人："他这篇论文真的写得好详细啊。"

嘉宾：

"是的，这些细节东西其实是需要去处理的。如果你不处理，写出来大家就觉得这个事情肯定也没有问题、可以这么做。有一些没写的部分，大家就会去追求——如果有些团队的组织密度没有那么足够，说'我自己就能把这个事情搞定'的话，就会有一部分人沉迷于小道消息，私下里打听这些东西到底是怎么处理的。如果你写出来，大家就不需要去找小道消息了。"

关于复现难度：

"我觉得 **infra** 上的事情，本质上没有什么难的，因为它本身就是个高度确定性的东西。但是如果你想自己提出来，那就对你本身分析问题的方式、对你的组织密度、团队氛围有比较高的要求。如果你整个 **infra** 团队比较好，你还是可以自己搞出来的；如果不行，你就尝试去 follow 一些比较好的工程实践。但如果这个事情都搞不定，那我觉得就不太应该了。"

6.7 RL Infra

主要挑战：大规模 **Sandbox** 管理

"RL 本身不太一样的点是它需要对大规模 sandbox 有比较细的要求。这算是一个更工程的东西。对于 RL 的 **task** 来说，每一个 **trajectory** 可能都对应一个它自己的 **Docker**，所以如何去管理大规模 **Docker** 是这块比较麻烦的问题。"

一个"比较 tricky 也比较有意思"的优化: Gradient Buffer 复用

"K3 会时不时给你蹦出来一些比较有意思的东西。"

背景: 完整的 RL 有好几部分 model:

- 至少一个主 **model** (policy) ;
- 一个 **reference model**;
- 主 model 还有对应的 **gradient** 和 **optimizer**;
- 如果还要做 **reward model**, 还会再多一个 model。

这会占用特别多 **memory** (一个 model 对 GPU 来说就是很大的 memory) 。

K3 的做法: 把 reference model 和 gradient buffer 的 memory 合并到一块。

为什么可以合并 (时序上互斥) :

1. **Reference model** 是没有梯度的——在求 objective 的过程中你是没有 gradient 的;
2. 等你有了 **gradient** 之后, **reference model** 就不会再用了。

论文中的表述大致是 "gradient buffer release for non-policy model forwarding"。

6.8 Inference Engine 的适配

前提: 主体的 inference optimization 是 vLLM / SGLang 这些专门讨论的内容。K3 论文这块讲的是: **K3 架构所带来的增量, 在现代推理引擎中如何处理。**

6.8.1 Linear Attention 的 Prefix Cache 难题

	Full Attention	Linear Attention
Cache 性质	每个位置都有一个 cache, 下一个位置的 cache 一定是上一个 cache 的简单增量	每一步会在上一步的 state 基础上做"复写"
Prefix Cache	简单很多	每一步的 prefix state 都不一样

两难:

- 如果**每一步都存起来** → linear attention 的好处 (省存储) 就完全没有了;
- 如果**完全不存** → 完全没办法享受 prefix cache 的好处。

目前的策略: 按 **block** 去切, 每个 **block** 做一次 **prefix caching**。

6.8.2 KV Cache 管理的异构性问题

问题: vLLM 这类引擎, **KV cache** 最简单的分配方式是不考虑"层和层之间异质性"的。

"所以从去年、前年开始, 如果想直接在 vLLM 里引入混合架构, 在 **infra** 上并不是一个容易的事情。而且因为现在 inference engine 越来越 heavy, 如果用一些简单的改法, 会把整个 inference engine 搞得特别特别乱。"

根本原因：

- **Full attention** 是按 **KV cache page** 管理的；
- 但 **linear attention** 或 **sliding window** 不能按 **page** 管理——因为它并不存在一个“对于每个 token 长久有效的 **KV cache** 产生方式”。

现在的 inference engine 可能已经有一些比较先进的管理方式了，但这仍然需要**特殊处理**，把不同的 attention pattern 结合起来。这对 vLLM 的设计来说，需要做一些兼容性处理。

6.8.3 其余部分

Decoding 的优化跟 prefill 或 training 不太一样，所以每个环节都需要额外的 kernel 配置。“我觉得这块都是可以去做的，而且应该是一个确定性比较高的事情。”

6.9 Evaluation

“没啥好讲的，就是好呗，特别好，在每个环节都好。”（K3 的结果数字摆在那里，没有太多可解读的。）

7. 观点与访谈精华

7.1 关于“没有范式创新”与忒修斯之船

主持人的感受：现在的模型训练好像没有一个巨大的所谓范式创新，但它把之前很多工作糅合起来，再去寻找一个可能的更优解。

嘉宾的回应（本集最出圈的一段）：

“我觉得问题不大，而且不完全是技术的原因。因为大家现在都叫 Transformer，但什么是 **Transformer**？其实是没有办法清晰定义的。

哲学上有个对应的概念就是**忒修斯之船**。2017 年这个船刚开始行驶的时候是 Transformer。但这个船刚开始是怎么拼起来的？它是 **Attention + Residual** 这些东西拼起来的，包括 stack layer 这种方式——它在 **Transformer** 之前其实也并不是一个全新的概念。

所以这个船刚建起来的时候，我们认为 Transformer 是一个 milestone。但这个船已经行驶了八九年，在行驶过程中我们会慢慢把某些部分换了、某些部分换了。现在到 **2026 年**，其实跟 **Transformer** 已经相似度很低很低了——你是否还把这个东西叫做 **Transformer**？

这当然是哲学上的问题。现在大家还是会叫，但如果历史的走向有些不同的变化，可能也就不叫了。”

7.2 K3 是一项什么样的工作？

“这个东西很难定义，因为它跟 report 一样，是一个偏综合性的工作。一般大家会把比较有创新性的单点突破专门拿出来讨论。中间比较大的就几点：

1. 注意力怎么设计
2. MoE 有没有更好的设计
3. 优化器
4. 中间的连接方式

这些如果放到 **ResNet** 时代，单拿出来可能就是一个架构了。但在 Transformer 时代它可能并不是这样——你得把这个东西综合起来，最后再把它跑出来，它才有意义。

之前大家做架构，在 ImageNet 上跑得足够高就是一个 milestone。但在这个时代，你想把实际的东西跑出来，要做的事情就特别特别多：你得做 scaling 的验证、把各个工程东西搞定、把 data 搞定，然后把这个 paper 或架构卖出来，大家才会 buy in。

当然从研究方面，我 personally 是不会太去关注的，因为我觉得架构的验证是有严格标准的，不需要把这些东西 couple 在一起。但从非技术角度，它现在是这样一个状态。”

7.3 K3 最 impressive 的地方

“个人而言，我 impress 的一点就是他们把激活搞得特别特别大，足足有 **100 Billion**，比我预期的还是要大一些的。”

这依赖什么能力？

“这个不依赖任何能力。具体定多少激活，本身不是一个技术性的东西，就看你想达到什么阶段的效果。我觉得 **Kimi K3** 还是比较有魄力——在已有开源模型的基础上要提升一个数量级，比上一代激活多了 3 倍多。我觉得这个决定是一个非技术性的决定，但这是要一定魄力的。”

杨植麟喜欢说“有概率的非共识”，K3 上能看到什么 different bets？

“我觉得也没啥，因为有价值的技术创新他们都单写了一个 paper 了。所以 K3 刚出来的时候，我主要就 surprise 这个 size，别的都符合我预期。”

7.4 关于“里程碑”与渐进式进步

“我其实的一个观点就是：科学研究是不存在阶跃性提升的，只是说大家习惯把技术渐进性提升的某些节点称为一个 milestone。

我一直是这么认为的。技术永远是慢慢提升，靠公司里的所有人、项目里所有人、包括整个业界的同学一块去推动的。所以我觉得从这个角度没有什么 milestone。”

关于 DeepSeek-V3 / R1 的类比：

“R1 这些东西我觉得是锦上添花。它最大的意义在于它是国内第一个把模型 scale 到一定程度、并且把 RL 全流程打通的。

在 K3 之前，可能最大的就是千问 72B。这其实是从模型 size 上一个比较大的飞跃，因为有 size 才有智能。所以 K3 你把激活多了 3 倍、把模型也扩大 3 倍，它就是下一个 level 的，这个没有任何问题。”

核心是 size？

“是有效的 scaling。这完全是一个靠团队来做起来的东西。因为把模型增大，如果 trivial 来看，我就把几个数字调大一点，就可以达到更大 size——但这不解决任何问题。从技术上讲，扩大模型 size 本身不是创新。你把这个东西做 work，中间才有创新。”

7.5 什么是“科学”的研究方式

“有人说 Kimi 的研究方式相对比较科学，你认同吗？”

"首先我是认可的。我觉得这主要来源于他们内部的管理方式。"

公开里大家都知道的是：**Kimi** 是公开里大家认可的最强调"模型内科"的一个团队。Kimi 一直尝试让 pretraining 变得更加科学，比如：

- 让不同行为的 **trace** 可以以更可靠的方式被获取到；
- 让模型的不稳定性、模型 **collapse** 的现象可以更明确地被归因。

我觉得这就是他们科学化的一个体现。"

"什么是不科学的？"

"我觉得不科学的方式是：从整体项目角度，你想去推一个东西、你想证明某个东西是有效的——这个东西本身不存在任何疑问，因为学术创新、学术论文管理是有明确标准的：你得有明确的 **ablation**，你得有足够的变量控制，你得用更科学的方式说明你这个东西是有效的，这个大家都认可。

那为什么不科学？不科学就是——基于某些利益需求，然后把科学的方式都干掉了。这就是不科学，不特指任何技术。

我一直强调的是：**Kimi** 这个团队非常团结，非常能把劲往一处使。如果大家是这样的 **motivation**，你的判断方式自然而然就是科学的。"

7.6 K3 vs DeepSeek-V4

"我觉得是不同层面吧。从模型定位、从 infra 上的区别、从模型架构层面，目前区别还是比较大的，完全也不是一个东西。

DeepSeek 现在出了两档：一档是 1.2T 的，另一档是更小的 size。**DeepSeek** 目前还是更强调性价比，他们那个 Flash 模型做得还是挺好的。

但 **Kimi** 目前主要的精力还是尝试去提升开源模型的能力上限，而不是去考虑性价比——也不是完全不考虑，至少说性价比不是最首要的考虑条件。当然也可以看到，现在 **K3** 的 API 相当贵，比别的模型贵多了。"

"照这个趋势发展，**Kimi** 和 **DeepSeek** 会分化吗？"

"我觉得技术层面的东西，只要大家不笨，都是知道什么是好、什么是不好的。技术上不存在太多 **debate**，好就是好，不好就是不好，这是有明确客观标准的。

但组织团队的走向取决于人的选择。对于组织而言、对于人而言，这个是比较模糊的。"

7.7 关于团队与组织

"新团队现在再去重做一个模型，还有时间窗口或机会吗？"

"如果这个团队足够好、团队氛围足够好、单兵作战能力足够强，是有机会的。但这个条件我觉得大概率是不存在的，所以后面的事情也就不存在。"

"为什么不存在？因为好的人已经进去了？"

"我觉得好的人和好的组织文化想达成，是有历史局限的——不是你想达成就能达成的。"

"氛围这个事情有点虚"

"我觉得也不虚。我觉得就是取决于 leader 的 taste，或者说 leader 的性格，是绝大多数影响到整个团队氛围的。"

"Kimi 有一个特点是，所有人好像都不怕杨植麟"

"对呀，为什么要怕呢？有道理你就.....谁有道理谁说了算嘛。"

(主持人补充：所以它是一个非常集体的工作，不依赖个人英雄主义。嘉宾：对的。)

7.8 对未来的判断

关于架构创新的空间 ("暴论")：

"我的暴论就是：大模型可能没有太本质的创新了，后面都是一些改良性的进步。"

能力会做到什么水平？

"重要的是大家去定义什么能力。只要能定义一个能力，就能去达到。因为之前定义的方式，比如强 coding、agent coding，这些东西清晰定义出来，大家就可以做到。这取决于大家怎么定义能力，或者从商业化角度大家注重什么能力。"

能达到 AGI 吗？

"我觉得 AGI 最大的问题就是它没有被 well-defined。"

"如果你定义 AGI 是具身、是真实物理世界交互，这个事情肯定 gap 是更大的。但如果还是在 language model 的 scope 里讨论问题，我觉得问题不大。"

模型会无限大吗？

"这个事情肯定是不可能的。首先，预训练的数据量——人类互联网上能集结到的所有信息量，这是有限的，这个事情不可能无限提升。

基于这个 case，从数学理论上，你想吃掉一个足够大的数据量，你的模型不可能是无限的，也没有必要是无限的。

这带来一个更宽的 bound（上限可能更高）和一个更窄的 bound（在什么情况下我们对模型在具体任务上的感知可能没有那么充分）。如果未来有更难的任务必须有更大的 size，可能就会退化到更宽的那个 bound 里。但无论如何不可能是无限的。"

"现在是 2.8T，以后呢？"

"闭源的肯定还是有更大的。如果想完全 match 闭源，肯定还是可以再打一些的。但更大的话，这个就不确定了。"

"你为什么去探索世界模型了？"

"我觉得这主要是一个 personal 的东西。因为我觉得大模型不存在太大的改进空间了，没有大的突破，就没有大的 credit。所以对个人来说，从职业生涯上讲，不是一个能做出太大贡献的领域了。

(世界模型) 不是一个 well-defined 的问题，这玩意问题更大。但我觉得至少是一个新的东西。"

8. 与延伸阅读

8.2 本次分享串联的工作清单

线性注意力线

1. Linear Attention（最原始形式）
2. **RetNet** — 衰减机制 + chunkwise recurrent（孙宇韬等）
3. **Mamba / Mamba2** — 位置无关衰减 → 位置有关衰减
4. **DeltaNet** — Delta Rule 提升上下文容量；Songlin Yang 解决并行化
5. **Gated DeltaNet** — 衰减 + Delta Rule 结合
6. **Kimi Linear / KDA** — channel-wise 衰减 + lower-bounded decay

高效架构线

7. **YOCO** — 全层共享一份 KV cache，计算与存储解耦
8. **YOCO-Universal** — Loop LM 只在 linear attention 段
9. **DeepSeek Sparse Attention (DSA)** — DeepSeek-3.2 / GLM-5
10. **Index Cache** — GLM 团队，跨层共享 sparse index

Attention 变体线

11. **MLA** — DeepSeek-V2/V3、K2、K3
12. **Gated Attention** — 千问（Qwen）团队
13. **Gated MLA** — K3 的结合

残差 / 连接线

14. **ResNet**（CVPR 版 & ICCV 版，pre-LN vs post-LN 之争）
15. **DenseNet** — 黄高
16. **HyperConnection** — 字节 Seed 团队
17. **MHC** — DeepSeek
18. **Attention Residual / Block Attention Residual**

MoE 线

19. **Latent MoE** — 英伟达团队
20. **Loss-Free Balancing**
21. **Expert Choice**
22. **Quantile Balancing** — 苏剑林博客 + K3 论文的工程实现

优化器 / 稳定性线

23. **Muon**
24. **Moonlight** — 首次在大规模训练引入 weight decay
25. **MuonClip / QK-Clip** — Kimi K2
26. **QK-Norm** — DeepSeek-V4、StepFun 等（配 GQA）

27. gpt-oss 的 Hard Clip → tanh Soft Clip → C2GLU

训练策略线

28. WSD (MiniCPM) vs Cosine Decay (K3 选后者)

29. NoPE in Hybrid Attention

后训练线

30. MiniLM — 微软亚研院董力团队, Reverse KL

31. On-Policy Distillation — Thinking Machines Lab 命名

32. EAGLE-3 / MTP

33. DeFlash — 连接 AR LM 与 Diffusion LM

Infra 线

34. DeepEP

35. DualPipe / DualPipe-V — DeepSeek-V3

36. Ring Attention / Zigzag Attention — Full Attention 的 CP

37. TileLang / TensorRT

8.3 可以直接记住的十条结论

1. 模型性能最主要的因素永远是参数量；架构创新对性能的提升微小，但对推理效率的影响是数量级的。
2. **Hybrid Attention** 从架构上是 **trade-off**，从效果上不是 **trade-off**——保持一定全注意力比例（约 3:1）可以无损甚至更好。
3. 模型推理开销只有三块：prefill 时间、decode 时间、KV cache 存储。
4. **chunkwise recurrent** 是所有现代线性注意力的计算基座。
5. 两个矩阵连乘拆开写时，中间一定要加 **normalization**——表达能力冗余，但优化性质不冗余。
6. 稳定性 > 一点点能力：能力好一点差一点都不如稳定性重要，因为训崩了整个 run 的 checkpoint 都废了。
7. 推理阶段"免费"的改进最容易被采纳；任何"让推理变慢换效果"的改进都会被质疑"为什么不直接扩 size"。
8. **RoPE** 带来的是 **recency bias**，它不带来长文能力，甚至损害长文能力——在 hybrid 架构下 full attention 用 NoPE 更优雅。
9. 架构选型会反过来决定 **infra** 的复杂度：K3 选 Latent MoE → 通信量小 → 用 shared expert 就能 overlap → 不需要 DualPipe 那样复杂的设计。
10. 有效的 **scaling** 才是创新：把数字调大不解决任何问题，把它做 work 中间才有创新。